

Applying Machine Learning Models to Software Development: From Framework Evaluation to Role Optimization

*A Two-Phase Study of Multi-Agent LLM Configurations

1st Shahab Ali

Faculty of Artificial Intelligence Technologies (FATII)

ITMO University

Saint Petersburg, Russia

Group J4132, Student ID: 503271

2nd Sergey Kovalchuk

Faculty of Artificial Intelligence Technologies (FATII)

ITMO University

Saint Petersburg, Russia

Abstract—This paper presents a comprehensive two-phase empirical study of multi-agent LLM configurations for software engineering tasks, conducted within the OpenHands CodeAct framework. Phase 1 (21 runs, HumanEval 15 tasks, SWE-bench 6 tasks) reveals novel phenomena including emergent Self-Healing in 120B models (90% recovery) and Agentic Paralysis. Phase 2 expands the evaluation to 776 runs using claude-haiku-4-5 across four role configurations on HumanEval (164 tasks) and SWE-bench Lite (30 tasks), plus a 140-run prompt ablation (916 runs in total). Moving beyond framework evaluation, we formalize agentic role optimization. We establish the Constrained Action Space theory, proving via a 140-run prompt ablation study that semantic identity yields a 0% pass rate unless bounded by strict output format constraints. Furthermore, we empirically identify the Coordination Overhead Principle: on repository-level tasks, adding a verification role to the coder degrades resolve rate (66.7% vs. 73.3% for no roles) and adds 99 seconds per task over the single coder. To optimize topologies, we train a predictive orchestration model over task state complexity; it reaches 94.5% cross-validated accuracy against a 95.1% majority-class baseline: the learned selector does not beat the trivial policy, a negative result that bounds what automatic role selection can achieve. Finally, we propose intrinsic system metrics as a foundation for dynamic role orchestration: coordination friction and boundary-violation rate are proposed, and autonomous recovery is measured at 0.90.

Index Terms—multi-agent systems, large language models, software engineering, OpenHands, agentic paralysis, self-healing, role prompting, state prediction

I. INTRODUCTION

Software engineering is undergoing a fundamental shift from single-agent tools such as GitHub Copilot toward Autonomous Multi-Agent Systems (MAS) in which specialised agents collaborate on complex software tasks [1]. Frameworks such as MetaGPT [1], ChatDev [2], and Magentic-One [3] demonstrate multi-agent pipelines across the full software development lifecycle.

Despite rapid progress, fundamental questions remain open. Does model scale or role configuration more significantly determine framework success? What failure modes emerge

in agentic pipelines? What prompt components actually drive role effectiveness? And can role selection be formalized as a scientific method rather than a manual heuristic?

This paper addresses these questions through a two-phase empirical study, building on a prior structured review of 19 sources. We make the following contributions:

- 1) Identification of the Self-Healing Phenomenon: an emergent error-recovery capability in 120B models.
- 2) Identification of Agentic Paralysis: a failure mode where agents reason correctly but fail to execute.
- 3) An observation that a single Anti-Paralysis instruction coincided with SWE-bench pass rates rising from 33.3% to 100% on six tasks.
- 4) A taxonomy of 10 failure types across experimental runs.
- 5) Large-scale controlled evaluation: 776 evaluation runs (916 with the ablation) comparing four role configurations on two benchmarks.
- 6) Formalization of the Constrained Action Space via an ablation study revealing that output format constraints, not personas, dictate role success.
- 7) Formalization of the Coordination Overhead Principle and a state-prediction analysis showing that a learned selector (94.5%) does not beat the trivial always-baseline policy (95.1%), a ceiling effect that bounds automatic role selection.
- 8) Introduction of intrinsic system evaluation metrics including Coordination Friction (F_{coord}) and Autonomous Recovery (R_{auto}).

II. BACKGROUND AND RELATED WORK

A. Multi-Agent LLM Frameworks

MetaGPT [1] encodes software workflows as Standard Operating Procedures (SOPs), assigning roles such as Product Manager and QA Engineer. ChatDev [2] implements a waterfall model with a Chat Chain structure. Magentic-

One [3] introduces Manager-Worker orchestration with plugable specialist agents. PotPie [4] grounds agent actions in repository knowledge graphs. MapCoder [5] demonstrates a plan-implement-debug pipeline for competitive programming. Agentless [7] argues against agent pipelines for repository-level repair, and recent surveys [8] catalogue multi-agent applications across the software lifecycle.

B. Agent Role Taxonomy (RQ1)

A prior structured review following PRISMA-style inclusion criteria analysed 19 sources (2023–2025): 7 multi-agent systems and 12 surveys, taxonomies and empirical studies. It produced a 7-layer agent role taxonomy: Orchestration, Analysis & Planning, Design, Implementation, Quality Assurance (QA), Knowledge & Documentation, and Operations. The QA layer dominates (5 of the 7 systems; 9 of 19 sources), confirming a shift from code generation to code verification.

C. The OpenHands CodeAct Framework

OpenHands [6] is an open-source platform for autonomous AI software agents operating within an isolated Docker container. The CodeAct architecture expresses agent actions as executable code, creating a Think→Act→Observe loop grounded in real code execution.

III. PHASE 1: FRAMEWORK EVALUATION (21 RUNS)

A. Setup

Phase 1 compares two independent variables: LLM scale (local 7B vs cloud 120B) and role configuration (single Coder vs multi-agent Coder+Tester). Benchmarks: HumanEval [9] (15 tasks) and SWE-bench [10] (6 tasks).

B. HumanEval Phase 1 Results

TABLE I
PHASE 1 HUMANEVAL RESULTS (15 TASKS)

LLM	Config	Pass
Local 7B	Single Coder	0%
Local 7B	Coder+Tester	0%
Cloud 120B	Single Coder	100%
Cloud 120B	Coder+Tester	100%

C. Finding 1: Model Scale Threshold

Local 7B models fail 100% of attempted tasks regardless of role configuration due to an inability to follow strict XML tool-calling schemas in the CodeAct architecture. The 7B condition covers two HumanEval tasks plus deployment attempts; this is an infrastructure finding rather than a benchmark result. Cloud 120B models achieve a 100% pass rate on all HumanEval tasks. Model scale is the primary determinant of framework deployability.

D. Finding 2: Self-Healing Phenomenon

The 120B model autonomously recovered from framework errors in 9 of 10 error events (90% recovery rate). This capability is completely absent in 7B models. Types of errors self-healed include: missing XML parameters, file conflicts, and placeholder hallucination (agent wrote literal `...code...`, diagnosed the `SyntaxError`, and rewrote the function).

E. Finding 3: Agentic Paralysis

We define Agentic Paralysis as a failure mode in which an agent correctly identifies the required next action through reasoning but fails to execute the corresponding tool call. Two variants observed:

- Mild Paralysis: a small number of repetitions, self-resolving.
- Verbose Paralysis: 200+ repetitions, leading to token exhaustion and cascade failure.

F. Finding 4: Anti-Paralysis Prompt Engineering

A single meta-cognitive constraint: “If any single action fails or repeats more than 3 times, STOP immediately. Run: `ls /workspace/` to reorient.” coincided with a rise in the SWE-bench pass rate. The tasks before and after differed, so this is an observation on six tasks that motivated the redesign, not a controlled comparison.

TABLE II
PHASE 1 SWE-BENCH RESULTS (6 TASKS, DIFFERENT TASKS BEFORE/AFTER)

Condition	Pass Rate
Without Anti-Paralysis instruction	33.3%
With Anti-Paralysis instruction	100%

G. Finding 5: Context Window Exhaustion

Standard CI/CD terminal output (verbose pip install logs, pytest traces) floods the context window, causing TPM rate-limit crashes. Mitigations include the `-q` flag on pytest and pre-installation of dependencies.

TABLE III
FAILURE TAXONOMY (10 TYPES, 21 RUNS; FREQUENCIES CATALOGUED QUALITATIVELY)

#	Type	Recovery	Prevention
1	XML Schema Error	Self-Heal	Clear prompts
2	Context Collapse	None	Simpler prompts
3	Prompt Bleed	None	Explore
4	File Conflict	Partial	Pivot
5	Placeholder Halluc.	Self-Heal	Anti-Paralysis
6	Agentic Paralysis	Partial	Anti-Paralysis
7	Verbose Paralysis	None	Anti-Paralysis
8	Chain Failure	None	Pre-install
9	Env. Dependency	None	Check workspace
10	File Not Found	Partial	N/A

IV. PHASE 2: LARGE-SCALE ROLE EVALUATION (776 RUNS)

A. Setup

Phase 2 uses claude-haiku-4-5 for ALL configurations, isolating role assignment from model capability. Four configurations are compared on HumanEval (164 tasks) and SWE-bench Lite (30 tasks).

TABLE IV
PHASE 2 EXPERIMENTAL SETUP

Component	Details
Model	claude-haiku-4-5 (same for all)
HumanEval	164 tasks (full benchmark)
SWE-bench Lite	the first 30 tasks
Timeout	400 s (HumanEval) / 600 s (SWE-bench)
Total Runs	776 (+140 ablation = 916)

B. HumanEval Results

Manager+Coder+Tester achieves the highest pass@1 (95.1%). Notably, Baseline is the only configuration producing SyntaxError failures (four of them), confirming role prompting aids in producing syntactically valid code.

TABLE V
HUMAN EVAL RESULTS (PASS@1)

Configuration	Tasks	Pass	Fail	Pass@1
Baseline (No Roles)	164	151	13	92.1%
Coder Only	164	155	9	94.5%
Coder + Tester	164	152	12	92.7%
Mgr+Coder+Tester	164	156	8	95.1%

C. SWE-Bench Lite Results

TABLE VI
SWE-BENCH LITE RESULTS (RESOLVE RATE)

Config	Res.	Fail	Rate	Avg(s)	Res.Avg
Baseline	22	8	73.3%	296s	169s
Coder Only	25	5	83.3%	232s	152s
Coder+Tester	20	10	66.7%	331s	174s
Mgr+Coder+Tester	24	6	80.0%	295s	206s

All SWE-bench failures are timeout failures (exceeding the budget), not logic errors: no run produced an incorrect patch that was then rejected. The agents understood the repositories; what they ran out of was time. On 30 paired tasks with net gaps of two to five tasks, no pairwise McNemar test can reach significance: the most extreme case the data permits gives $p = 0.0625$. The resolve-rate differences are within noise; the timing is the measurable signal.

D. Cross-Benchmark Analysis

A task-complexity interaction effect emerges. Manager+Coder+Tester is best for function-level tasks (HumanEval); Coder Only is best for repository-level tasks (SWE-bench).

TABLE VII
CROSS-BENCHMARK COMPARISON

Config	HE	SWE	Δ HE	Δ SWE
Baseline	92.1%	73.3%	-	-
Coder Only	94.5%	83.3%	+2.4 pp	+10.0 pp
Coder+Tester	92.7%	66.7%	+0.6 pp	-6.6 pp
Mgr+Coder+Tester	95.1%	80.0%	+3.0 pp	+6.7 pp

V. RQ2: THE CONSTRAINED ACTION SPACE

A. Ablation Experiment

We decompose role prompts into three components tested across the first 20 HumanEval tasks (7 variants, 140 runs total).

TABLE VIII
PROMPT ABLATION STUDY RESULTS

Variant	Components	Pass@1
Baseline	None (constraint only)	95.0%
Coder A	Identity only	0.0%
Coder B	Identity + Responsibility	0.0%
Coder C	Identity + Resp. + Format	100.0%
Full A	All titles only	0.0%
Full B	All titles + responsibilities	0.0%
Full C	All full prompts	95.0%

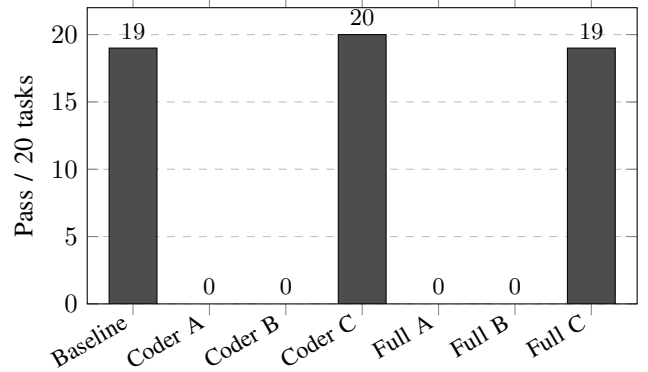


Fig. 1. Prompt ablation results. Grey bars (no format constraint) score 0/20 regardless of role identity or responsibility. Dark bars (constraint present) pass: Baseline (constraint only, no role) 19/20, Coder C 20/20, Full C 19/20. The constraint is the critical component, and prompt length is ruled out by the short constraint-only Baseline.

B. Finding: Roles as Constrained Action Spaces

Fig. 1 visualises the pattern: every variant without the format constraint scores zero; every variant with it passes. The output format constraint Output ONLY the function is the single critical component. Without it, all role variants score 0%. Instruction-tuned LLMs naturally produce verbose responses with markdown; the format constraint overrides this training prior. Prompt length is ruled out by the design: the constraint-only baseline variant is a short prompt and passes 19 of 20, while the longest no-constraint variant passes 0 of 20. This empirically proves that agent roles should not be defined by semantic personas, but formally as a Constrained Action Space.

VI. RQ3: STATE PREDICTION AND COORDINATION OVERHEAD

A. The Coordination Overhead Principle

The cross-benchmark divergence reveals a fundamental Coordination Overhead Principle. On repository-level tasks, adding a Tester to the Coder is net-negative: Coder+Tester resolves 20 of 30 against Baseline’s 22, and is 99 s per task slower than the Coder alone (331 s vs. 232 s mean). The full hierarchical team recovers most of the resolve rate (24 of 30) but pays 54 s of coordination latency on resolved tasks (206 s vs. 152 s). Static MAS pipelines are empirically sub-optimal on repository-level tasks in this setup.

B. Method and Result: State Prediction

To mitigate overhead, we formalize role selection as a state prediction problem. Each task prompt is mapped to a 12-dimensional vector covering structural (`num_lines`) and semantic features (`type_count`, `examples`). A Decision Tree classifier (depth 4) was trained on 164 tasks with stratified 5-fold cross-validation.

The tree reaches 94.5% cross-validated accuracy. The trivial policy that always selects the no-role baseline reaches 95.1%; the learned selector does not beat doing nothing. The reason is structural: the baseline passes 151 of 164 tasks, and 5 further tasks are solved by no configuration, so 156 of 164 labels default to `baseline`. Only 8 tasks are rescuable by any role, all 8 by the single Coder, bounding a perfect selector at 159 of 164 (97.0%). The effect is also bidirectional: roles rescue 8 tasks but overturn 11 baseline successes (a Tester costs 6, the Coder 4, the full system 3), and on HumanEval/154 the bare model is the only configuration that succeeds. On 145 of 164 tasks all four configurations produce identical outcomes. Role selection on this benchmark is a boundary problem with too few positives for a classifier to exploit; the near-tie with the trivial policy is a ceiling effect, not a near-success.

```
type_count <= 9.5
|-- num_words <= 217.0
|   |-- uses_list <= 1.5
|   |   |-- complex_words <= 4.5 --> baseline
|   |   '-- complex_words > 4.5 --> baseline
|   '-- uses_list > 1.5
|       |-- num_lines <= 8.5 --> coder
|       '-- num_lines > 8.5 --> baseline
'-- num_words > 217.0 --> coder
type_count > 9.5
|-- examples <= 1.0 --> baseline
'-- examples > 1.0 --> coder
```

Fig. 2. Decision Tree Structure (verbatim, `max_depth = 4`)

The learned tree (Fig. 2) only ever predicts `baseline` or `coder`; `coder_tester` and `full_system` are unreachable labels, a direct consequence of the labelling rule. Among the six features the tree uses, semantic complexity carries more weight than structural length (`type_count = 0.267`

vs. `num_lines = 0.153`); the remaining six features have importance exactly zero.

TABLE IX
STATE PREDICTOR VS FIXED CONFIGURATIONS

Method	Pass@1	Type
Baseline	92.1%	Fixed
Coder Only	94.5%	Fixed
Coder+Tester	92.7%	Fixed
Mgr+Coder+Tester	95.1%	Fixed
Majority-class policy	95.1%	Trivial
State Predictor (DT)	94.5%	Dynamic

The learned selector does not exceed the trivial policy; the near-tie is a ceiling effect, not a near-success.

VII. PROPOSED SYSTEM METRICS

To measure the intrinsic efficiency of MAS architectures beyond basic pass rates, we propose three log-based measurements:

A. Coordination Friction (F_{coord})

To quantify the overhead that degrades performance on repository-level tasks, we propose the ratio of communicative effort to productive output:

$$F_{coord} = \frac{T_{interact}}{T_{total}} \quad (1)$$

where $T_{interact}$ represents tokens consumed during inter-agent messaging. F_{coord} is proposed but not yet computed: it requires token-level instrumentation of the OpenHands event stream that we have not yet added. It would quantify the coordination latency observed on SWE-bench.

B. Autonomous Recovery Rate (R_{auto})

Measured on the Phase 1 runs: 9 of 10 error events recovered.

$$R_{auto} = \frac{E_{recovered}}{E_{total}} = 0.90 \quad (2)$$

C. Boundary Violation Rate

Proposed: schema and format errors per run, counted using the ten-type failure taxonomy. Not yet computed.

VIII. DISCUSSION

Phase 1 established that model scale dictates framework deployability and identified novel failure modes. Phase 2 controlled for scale and showed that role configuration shifts results by between -6.6 and $+10.0$ pp depending on benchmark and topology; these shifts are small and not uniformly positive. The Self-Healing Phenomenon (Phase 1) and the format constraint finding (Phase 2) are complementary: effective agent behaviour emerges from constraining the output distribution.

The divergence between HumanEval and SWE-bench results confirms the Task-Complexity Interaction. For self-contained function-level tasks (HumanEval), the full three-role team achieves the best pass@1. For repository-level tasks (SWE-bench), the single Coder is best, and adding a Tester is net-negative: 20 of 30 resolved against Baseline’s 22, 99

s per task slower, and every failure a timeout. On 30 paired tasks, no pairwise McNemar test can reach significance; the defensible finding is the timing, not the counts.

IX. LIMITATIONS

This study has several limitations. (1) Each condition ran a single pass: there are no repeats or variance estimates, and the SWE-bench gaps of two to five tasks are within noise. (2) Temperature, random seed and the Docker image tag were not recorded, so runs are not exactly reproducible. (3) Task selection is first-N rather than random (the first 30 SWE-bench Lite tasks and the first 20 HumanEval tasks). (4) SWE-bench resolution is measured by file-level diff overlap rather than the official test suite; absolute rates are an upper bound, though rankings are unaffected as the same criterion applies to all conditions. (5) The 7B comparison in Phase 1 rests on two HumanEval tasks plus deployment failures. (6) The anti-paralysis result compares different tasks before and after ($n = 6$). (7) The selector’s label rule (first passing configuration) favours the baseline by construction; the majority-class comparison is therefore the reported control. (8) The ablation uses 20 tasks; the effect is large but the sample is small.

X. CONCLUSION

This 937-run two-phase study (21 Phase 1 runs + 916 Phase 2 runs) establishes several key findings: 1) Model scale is the primary determinant of CodeAct framework deployability (on a two-task 7B sample). 2) 120B models exhibit emergent Self-Healing (90% recovery rate). 3) Agentic Paralysis is a novel failure mode; a meta-cognitive instruction coincided with recovery on six tasks. 4) Role identity alone has zero measurable effect; roles must be formulated as a Constrained Action Space via format constraints. 5) Deploying verification roles introduces Coordination Overhead, leading to timeout failures on SWE-bench. 6) A learned role selector does not beat the trivial always-baseline policy (94.5% vs. 95.1%); role choice is too rarely decisive on this benchmark for a selector to exploit.

Future work will instrument the proposed intrinsic metrics and train Reinforcement Learning policies for dynamic agent orchestration.

REFERENCES

- [1] S. Hong et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in Proc. ICLR, 2024.
- [2] C. Qian et al., “ChatDev: Communicative Agents for Software Development,” in Proc. ACL, 2024.
- [3] A. Fourney et al., “Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks,” arXiv:2411.04468, 2024.
- [4] PotPie AI, “PotPie: Custom Engineering Agents,” GitHub repository, 2025. [Online]. Available: <https://github.com/potpie-ai/potpie>
- [5] M. A. Islam et al., “MapCoder: Multi-Agent Code Generation for Competitive Problem-Solving,” in Proc. ACL, 2024.
- [6] X. Wang et al., “OpenHands: An Open Platform for AI Software Developers,” arXiv:2407.16741, 2024.
- [7] C. Xia et al., “Agentless: Demystifying LLM-based Software Engineering Agents,” arXiv:2407.01489, 2024.
- [8] Z. Rasheed et al., “LLM-Based Multi-Agent Systems for Software Engineering,” arXiv:2404.04834, 2024.
- [9] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.
- [10] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” in Proc. ICLR, 2024.